UTN
Facultad Regional Córdoba

TUP

Tecnicatura Universitaria en Programación

LABORATORIO DE COMPUTACIÓN I

Unidad Temática N°3:
Consultas

Guía Práctica
1° Año – 1° Cuatrimestre

CC BY ND NC

Índice

Sentencias de Recuperación de Datos 2

Problema 4 (Continuación de Unidad 2) 2
- a) Test de Rango: Between 2
- b) Test de correspondencia con patrón de búsqueda: LIKE 2
- c) Test de valor nulo: IS NULL 3
- d) Test de pertenencia a un conjunto: IN 3
- e) Varias condiciones de búsqueda 3
- f) Consultas con varias tablas 4

Problema 5: Resuelva utilizando JOIN EXPLÍCITO 5

Problema 6: Ejercicios Combinados Extras 9

Solución del Problema 4 (Continuación de Unidad 2) 12
- a) Test de Rango: Between 12
- b) Test de correspondencia con patrón de búsqueda: LIKE 12
- c) Test de valor nulo: IS NULL 13
- d) Test de pertenencia a un conjunto: IN 13
- e) Varias condiciones de búsqueda 14
- f) Consultas con filtros combinados 15

Solución del Problema 5: Resuelva utilizando JOIN EXPLÍCITO 17

Sentencias de Recuperación de Datos

Problema 4 (Continuación de Unidad 2)

El vivero de la ciudad posee una base de datos cuyo diagrama se presenta a continuación. El administrador del vivero requiere obtener información precisa y actualizada para tomar decisiones informadas en su trabajo diario.

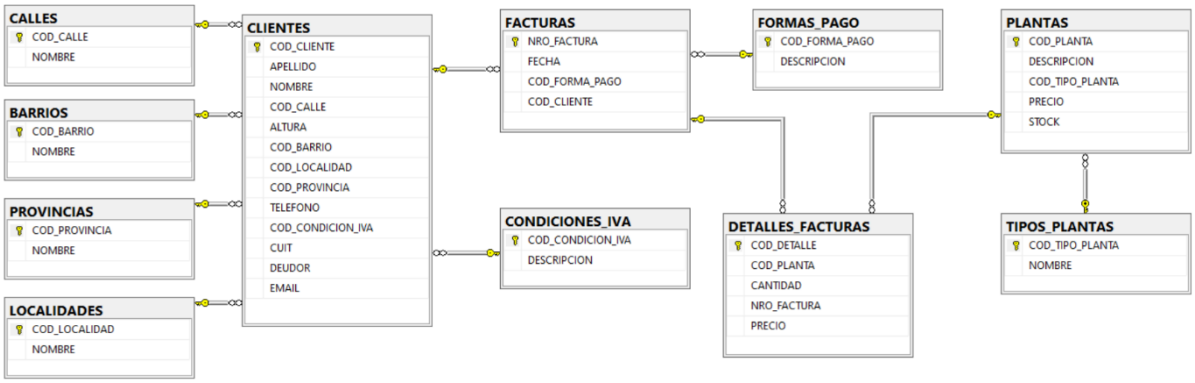

Imagen 1: Elaboración propia.

Para ello abra la base de datos VIVERO_FENIX y resuelva las consultas que se piden a continuación.

a) Test de Rango: Between

1. Mostrar la descripción y el precio de las plantas, cuyo precio está entre 300 y 700, ordenadas por precio de forma ascendente:
2. Mostrar la descripción, stock y precio de las plantas cuyo stock oscile entre 20 y 50. Ordenar por precios en forma descendente.
3. Listar el nombre y el número de teléfono de los clientes cuyo número de calle está entre 1 y 100, ordenados alfabéticamente por apellido
4. Listar los datos de las facturas emitidas entre el 1 de enero de 2022 y el 31 de marzo de 2022.
5. Listar los datos de las facturas emitidas en el año 2019.

b) Test de correspondencia con patrón de búsqueda: LIKE

6. Todos los clientes cuyo apellido comience con “f”.
7. Todos los clientes cuyo nombre contenga una “u”.
8. Todos los clientes cuyo apellido no comience con letras que van de la “a” a la “c”.

9. Todos los clientes cuyo apellido termine con “ez”.
10. Listar todos los clientes cuyo apellido no comience con “p”.
11. Ídem al anterior pero que no comience ni con “p” ni con “z”.

c) Test de valor nulo: IS NULL

12. Mostrar todos los clientes que no tengan teléfono.
13. Listar apellido, nombre, e-mail de los clientes cuyas direcciones de mail sea conocidas.
14. Listar apellido, nombre, teléfono de los clientes cuyo teléfono sea conocido.

d) Test de pertenencia a un conjunto: IN

15. Listar las facturas de los clientes 1,5,7 y 10
16. Listar todos los datos de las plantas que la descripción sea alegría, rosa y pino.
17. Listar apellido, nombre, teléfono de los clientes cuyo apellidos no sea Alvarez y Perez.

e) Varias condiciones de búsqueda

18. Listar todas las plantas que comienzan con R y precio superior a $7.
19. Mostrar todas las plantas cuyo precio sea menor a $ 150 o mayor a $ 800, Mostrar también su precio con un aumento del 10%, ordenar por precio.
20. Listar apellido, nombre, teléfono e e-mail de los clientes cuyos direcciones de mail o teléfono sean conocidos.
21. Listar los datos de las plantas cuya descripción no comiencen con “m” ni con “n” ni con “p” o su precio sea mayor a 40 con una valoración de stock menor a $ 1.0000.
22. Mostrar todas las plantas cuyo stock sea mayor a 15 y menor a 30 o bien su precio aumentado en un 15% esté entre 150 y 500. Ordenar por descripción.

f) Consultas con varias tablas

23. Listado de clientes mostrando apellido y nombre, calle y número.
24. Listado de clientes mostrando apellido, nombre y localidad a la que pertenece.
25. Listado de clientes mostrando apellido, nombre y barrio al que pertenece.
26. Listar todas las facturas abonadas en efectivo, ordenada por código de cliente.
27. Listar todas plantas donde el tipo de planta sea Flores.
28. Listar todos los clientes cuya localidad sea Jesús María.
29. Listar todos los registros completos de clientes cuyo IVA sea igual a MONOTRIBUTISTA, ordenado por localidad.
30. Listar todos los datos de los clientes que viven en la localidad de Córdoba o su apellido comience con la letra ‘A’.
31. Listar clientes que viven en la localidad de CARLOS PAZ, ALTA GRACIA y ADELIA MARIA.
32. Mostrar todos los clientes que no viven en las localidades de AREQUITO, CORDOBA y MERLO.
33. Mostrar nombre, apellido y barrio de los clientes de Cba. Capital o de aquellos cuyos apellidos no comiencen con letras de la ‘D’ a la ‘P’. Ordenar por barrio.
34. Listar el detalle de la factura calculando la cantidad por el precio.
35. Listar todos los clientes que vivan en provincias que comiencen con 'Co' y que sean CONSUMIDOR FINAL.
36. Listar todos los clientes que viven en barrio General Paz, Centro y que además posean teléfono.

Problema 5: Resuelva utilizando JOIN EXPLÍCITO

Los usuarios finales de la librería mayorista necesitan obtener información para el funcionamiento del negocio y la toma de decisiones. Se vuelve a incluir la estructura de la base de datos:

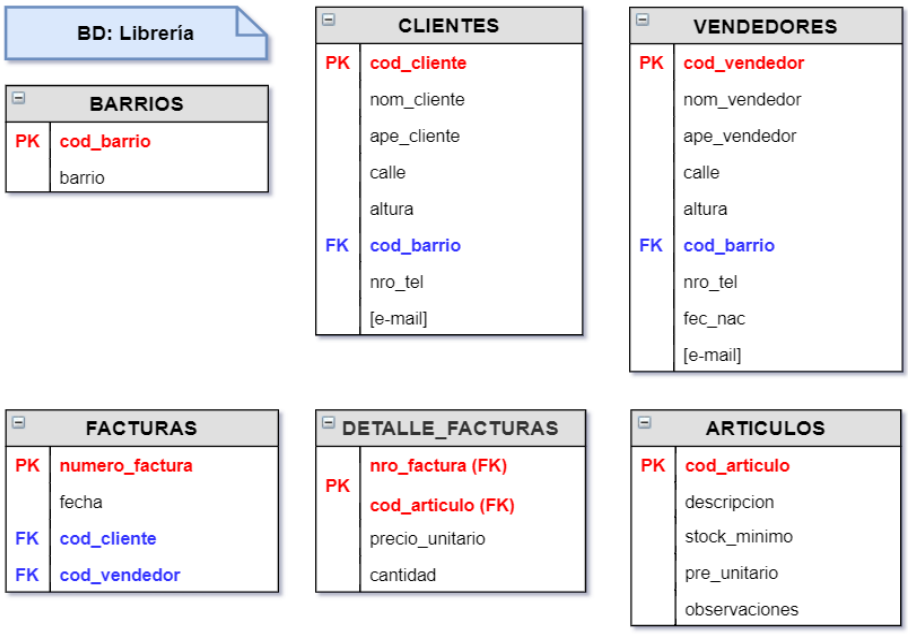

Imagen 2: Elaboración propia.

Resuelva utilizando JOIN EXPLÍCITO los siguientes reportes:

1. Se quiere tener un listado de los vendedores y los barrios donde viven, ***incluso aquellos vendedores donde no tengan un barrio asignado***.

Para resolver este problema paso a paso, es importante leer cuidadosamente la consigna e interpretar lo que se solicita. En una lectura rápida, podemos identificar los siguientes puntos clave: *obtener un listado de los vendedores y los barrios donde viven,* ***incluso aquellos vendedores donde no tengan un barrio asignado***.

En primer lugar, necesitaremos combinar la tabla de vendedores con la tabla de barrios utilizando una operación de **JOIN**. Sin embargo, en este caso, debemos utilizar un **LEFT JOIN** en lugar de un **JOIN** común.

La diferencia radica en que un **JOIN** común, también conocido como **INNER JOIN**, **devuelve únicamente los registros que tienen coincidencias en ambas tablas según la condición de unión especificada**. En otras palabras, solo se mostrarán los registros que tengan un vendedor asociado y que estén presentes en ambas tablas.

Por otro lado, un **LEFT JOIN** **devuelve todos los registros de la tabla izquierda** (en este caso, la tabla de vendedores) **y los registros coincidentes de la tabla derecha** (en este caso, la tabla de barrios) según la condición de unión. **Si no hay una coincidencia en la tabla derecha, se mostrará NULL** en las columnas correspondientes.

Al utilizar un **LEFT JOIN** en este escenario, garantizamos que todos los barrios, incluso aquellos sin vendedores asociados, estén incluidos en el resultado. Esto es importante si necesitamos mostrar una lista completa de todos los barrios, independientemente de si hay vendedores viviendo allí o no.

Aquí tienes los pasos para resolver esta consigna:

Paso 1: Identificar las columnas necesarias

- Vendedores: nom_vendedor, ape_vendedor, cod_barrio
- Barrios: cod_barrio, barrio

Paso 2: Escribir la consulta SQL

```
SELECT V.NOM_VENDEDOR + ' ' + V.APE_VENDEDOR 'VENDEDOR',
       B.BARRIO 'BARRIO'
  FROM VENDEDORES V
  LEFT JOIN BARRIOS B ON V.COD_BARRIO = B.COD_BARRIO
```

Paso 3: Explicación de la consulta

- Utilizamos la cláusula SELECT para seleccionar las columnas que deseamos mostrar en el resultado.
- Renombramos las columnas para que sean más descriptivas (por ejemplo, "V.NOM_VENDEDOR y V.APE_VENDEDOR" para el nombre del vendedor y "B.BARRIO" para el nombre del barrio).
- Especificamos las tablas que estamos utilizando en la consulta: "VENDEDORES" y "BARRIOS".
- Utilizamos la cláusula **LEFT JOIN** para unir las tablas, asegurándonos de incluir todos los registros de la tabla de vendedores, incluso si no hay coincidencias en la tabla de barrios.
- **Establecemos la condición de unión utilizando la columna "cod_barrio" tanto en la tabla de vendedores como en la tabla de barrios**.

Con esta consulta actualizada, obtendremos un listado de todos los vendedores y los barrios donde viven, incluyendo los barrios sin vendedores asociados. Si no hay un vendedor viviendo en un barrio, se mostrará NULL en la columna del barrio.

```sql
SELECT V.NOM_VENDEDOR + ' ' + V.APE_VENDEDOR 'VENDEDOR',
       B.BARRIO 'BARRIO'
  FROM VENDEDORES V
  LEFT JOIN BARRIOS B ON V.COD_BARRIO = B.COD_BARRIO;
```

158 %

Results | Messages

	VENDEDOR	BARRIO
1	Martín Carrizo	ALTO ALBERDI
2	Mariela Ledesma	GENERAL PAZ
3	Alejandro Lopez	OBSERVATORIO
4	Marcelo Miranda	CENTRO
5	Gabriel Monti	JARDÍN
6	Susana Juarez	SAN VICENTE
7	Facundo Funes	NULL
8	Florencia Almada	NULL
9	Mariela Suarez	NULL

Imagen 3: Elaboración propia.

IMPORATNTE: Una pregunta que uno podemos hacer es la siguiente:

🤔 *¿Es posible utilizar un **RIGHT JOIN** en lugar de un **LEFT JOIN** en esta situación?*

😃 *¡La respuesta es, SI!*

Veamos, un **RIGHT JOIN** funciona de manera similar a un **LEFT JOIN**, pero ***invierte el orden de las tablas***. En lugar de incluir todos los registros de la tabla izquierda (Vendedores en este caso), incluirá todos los registros de la tabla derecha (Barrios) y los registros coincidentes de la tabla izquierda.

Aquí tienes la consulta actualizada utilizando un **RIGHT JOIN**:

```sql
SELECT V.NOM_VENDEDOR + ' ' + V.APE_VENDEDOR 'VENDEDOR',
       B.BARRIO 'BARRIO'
  FROM BARRIOS B
 RIGHT JOIN VENDEDORES V ON B.COD_BARRIO = V.COD_BARRIO
```

En esta consulta, hemos invertido el orden de las tablas, colocando la tabla "Barrios" a la izquierda del **RIGHT JOIN** y la tabla "Vendedores" a la derecha. Esto nos permitirá mostrar todos los barrios, incluyendo aquellos donde no haya vendedores viviendo, junto con los vendedores asociados a los barrios correspondientes.

Ten en cuenta que, en la práctica, el uso de **RIGHT JOIN** no es tan común como el uso de **LEFT JOIN**. En la mayoría de los casos, se prefiere utilizar **LEFT JOIN** para mantener una estructura de consulta más intuitiva y legible. Sin embargo, ambos **JOINs** pueden lograr el mismo resultado en este caso específico.

Resuelva utilizando JOIN EXPLÍCITO los siguientes reportes:

2. Genere un reporte con los datos de la facturación (datos de las facturas incluidos los del vendedor y cliente) de los años 2006, 2007, 2009 y 2012.
3. Liste los datos de la facturación, de los artículos y de la venta de las facturas correspondientes al mes pasado.
4. Emita un listado con los datos del vendedor y las ventas que ha realizado el año pasado. Muestre los vendedores aun así no tengan ventas registradas en el año solicitado.
5. Liste descripción, cantidad e importe; aun para aquellos artículos que no registran ventas.
6. Genere un reporte con los datos de la facturación (datos de las facturas incluidos los del vendedor y cliente) y de la venta (incluido el importe); para las ventas de febrero y marzo de los años 2006 y 2007 y siempre que el artículo empiece con letras que van de la “a” a la “m”. Ordene por fecha, cliente y artículo.
7. Liste código de cliente, nombre, fecha y factura para las ventas del año 2007. Muestre los clientes hayan comprado o no en ese año.
8. Se quiere saber los artículos que compró la cliente Pilar Morales en el año pasado. Liste cliente, artículo e importe.
9. Se quiere saber los artículos que compraron los clientes cuyos apellidos empiezan con “p”. Liste cliente, artículo, cantidad e importe. Ordene por cliente y artículo, esté en forma descendente. Rotule como CLIENTE, ARTICULO, CANTIDAD, IMPORTE.

Problema 6: Ejercicios Combinados Extras

Los usuarios finales de la librería mayorista necesitan obtener información para el funcionamiento del negocio y la toma de decisiones. Se vuelve a incluir la estructura de la base de datos:

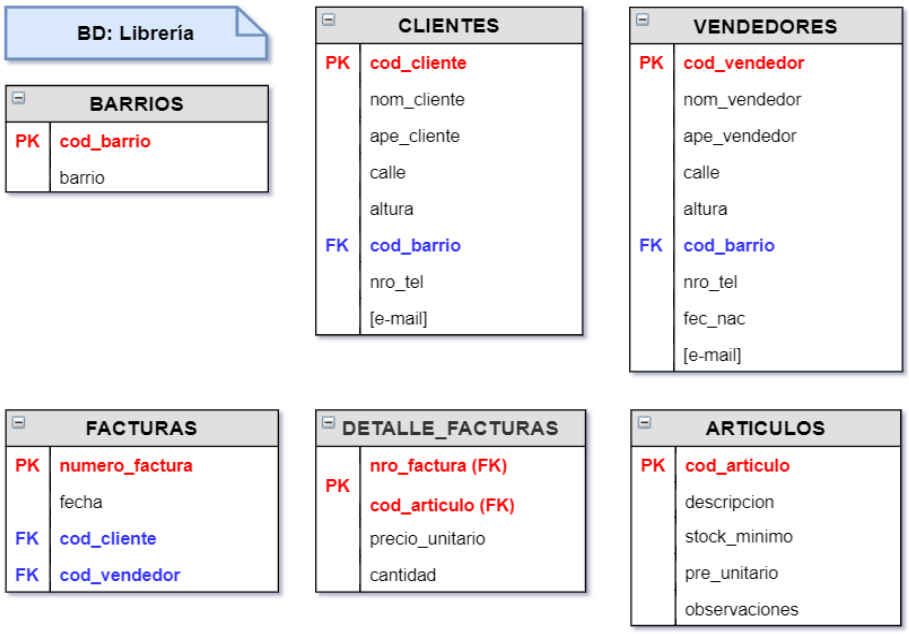

Imagen 3: Elaboración propia.

Resuelva los siguientes reportes:

1. Se necesita mostrar el código, nombre, apellido (todo el apellido en mayúsculas) y dirección (calle y altura en una sola columna; para la altura utilice una función de conversión) de todos los clientes cuyo nombre comience con “C” y cuyo apellido termine con “Z”. Rotule como CÓDIGO DE CLIENTE, NOMBRE, DIRECCIÓN.
2. Ídem al anterior pero el apellido comience con letras que van de la “D” a la “L” y cuyo nombre no comience con letras que van de la “A” a la “G”.
3. Muestre los datos de los vendedores (apellido todo en mayúsculas y en la misma columna que el nombre) cuyo nombre no contenga “Z”, haya nacido en la década del 70 y que haya realizado ventas el mes pasado.
4. Mostrar las facturas realizadas entre el 1/1/2007 y el 1/5/2009 y cuyos códigos de vendedor sean 1, 3 y 4 o bien entre el 1/1/2010 y el 1/5/2011 y cuyos códigos de vendedor sean 2 y 4. Mostrar la fecha en formato Dia, Mes, y Año (en ese orden y sin la hora)

5. Se quiere saber el subtotal de todos los artículos vendidos, para ello listar el artículo y multiplicar la cantidad por el precio unitario de venta; mostrar el subtotal redondeado a dos decimales (o buscar la forma de darle formato apropiado). Ordenar alfabéticamente por artículo y cuyo subtotal mayor aparezca primero. No muestre datos duplicados.
6. Muestre las ventas (tabla detalle_facturas) de los artículos cuyo precio unitario actual sea mayor o igual a 50 o cuyos códigos de artículos no sea uno de los siguientes: 2,5, 6, 8, 10. En ambos casos los precios unitarios a los que fueron vendidos oscilan entre 10 y 100.
7. Listar todos los datos de los artículos cuyo stock mínimo sea superior a 10 o cuyo precio sea inferior a 20. En ambos casos su descripción no debe comenzar con las letras “p” ni la letra “r”.
8. Listar los datos de los vendedores nacidos en febrero, abril, mayo o septiembre.
9. Liste número de factura, fecha de venta y vendedor (apellido y nombre), para los casos en que los códigos del cliente van del 2 al 6. Ordene por vendedor y fecha, ambas en forma descendente.
10. Emitir un reporte con los datos de la factura del cliente y del vendedor de aquellas facturas confeccionadas entre el primero de febrero del 2008 y el primero de marzo del 2010 y que el apellido del cliente no contenga “C”.
11. Listar los datos de la factura, los del artículo y el importe (precio por cantidad); para las facturas emitidas en el 2010, 2015 y 2017 y la descripción no comience con “R”. Ordene por número de factura e importe, en forma descendente. Rotule.
12. Se quiere saber qué artículos se vendieron, siempre que el precio unitario sin iva al que fue vendido no esté entre $10 y $50. Rotule.
13. Liste todos los datos de la factura (vendedor, cliente, artículo, incluidos los datos de la venta: cantidad, precio y subtotal); emitidas a clientes con teléfonos o direcciones de e-mail conocidas de aquellas facturas cuyo importe haya sido superior a $250. Agregue rótulos y ordene el listado para darle mejor presentación.
14. Se quiere saber a qué cliente, de qué barrio, vendedor y en qué fecha se les vendió con los siguientes nros. de factura: 12, 18, 1, 3, 35, 26 y 29.¿El barrio del cliente es el mismo que el barrio del vendedor que les vendió?

15. Emitir un reporte para informar qué artículos se vendieron, en las facturas cuyos números no estén entre 17 y 136. Liste la descripción, cantidad e importe (Importe=cantidad*pre_unitario). Ordenar por descripción y cantidad. No muestre las filas con valores duplicados
16. Listar los datos de las facturas (cliente, artículo, incluidos los datos de la venta incluido el importe) emitidas a los clientes cuyos apellidos comiencen con letras que van de la “l” a “s” o los artículos vendidos que tengan descripciones que comiencen con las mismas letras. Ordenar el listado.
17. Realizar un reporte de los artículos que se vendieron en lo que va del año. (Muestre los datos que sean significativos para el usuario del sistema usando rótulos para que sea más legible y que los artículos no se muestren repetidos).
18. Se quiere saber a qué clientes se les vendió el año pasado, qué vendedor le realizó la venta, y qué artículos compró, siempre que el vendedor que les vendió sea menor de 35 años.
19. El usuario de este sistema necesita ver el listado de facturas, de aquellos artículos cuyos precios unitarios a los que fueron vendidos estén entre 50 y 100 y de aquellos vendedores cuyo apellido no comience con letras que van de la “l” a la “m”. Ordenado por vendedor, fecha e importe.
20. Se desea emitir un listado de clientes que compraron en enero, además saber qué compraron cuánto gastaron (mostrar los datos en forma conveniente)
21. Emitir un reporte de artículos vendidos en el 2010 a qué precios se vendieron y qué precio tienen hoy. Mostrar el porcentaje de incremento.
22. Listar los vendedores que hace 10 años les vendieron a clientes cuyos nombres o apellidos comienzan con "C".
23. El encargado de la librería necesita tener información sobre los artículos que se vendían a menos de $20 antes del 2015. Mostrar los datos que se consideren relevantes para el encargado, formatear, rotular y ordenar.

Solución del Problema 4 (Continuación de Unidad 2)

a) Test de Rango: Between

1.
```
SELECT DESCRIPCION, PRECIO
  FROM PLANTAS
 WHERE PRECIO BETWEEN 300 AND 700
 ORDER BY PRECIO ASC;
```

2.
```
SELECT DESCRIPCION, STOCK, PRECIO
  FROM PLANTAS
 WHERE STOCK BETWEEN 20 AND 50
 ORDER BY PRECIO DESC;
```

3.
```
SELECT APELLIDO, NOMBRE, TELEFONO
  FROM CLIENTES
 WHERE COD_CALLE BETWEEN 1 AND 100
 ORDER BY APELLIDO ASC;
```

4.
```
SELECT *
  FROM FACTURAS
 WHERE FECHA BETWEEN '01/01/2022' AND '31/03/2022';
```

5.
```
SELECT *
  FROM FACTURAS
 WHERE FECHA BETWEEN '01/01/2019' AND '31/12/2019';
```

b) Test de correspondencia con patrón de búsqueda: LIKE

6.
```
SELECT *
  FROM CLIENTES
 WHERE APELLIDO LIKE 'f%';
```

7.
```
SELECT *
  FROM CLIENTES
 WHERE NOMBRE LIKE '%u%';
```

8.
```
SELECT *
  FROM CLIENTES
 WHERE APELLIDO NOT LIKE '[a-c]%';
```

9.
```
SELECT *
  FROM CLIENTES
 WHERE APELLIDO LIKE '%ez';
```

10.
```
SELECT *
  FROM CLIENTES
 WHERE APELLIDO NOT LIKE 'p%';
```

11.
```
SELECT *
  FROM CLIENTES
 WHERE APELLIDO NOT LIKE '[pz]%';
```

c) Test de valor nulo: IS NULL

12.
```
SELECT *
  FROM clientes
 WHERE telefono IS NULL;
```

13.
```
SELECT apellido, nombre, email
  FROM clientes
 WHERE email IS NOT NULL;
```

14.
```
SELECT apellido, nombre, telefono
  FROM clientes
 WHERE telefono IS NOT NULL;
```

d) Test de pertenencia a un conjunto: IN

15.
```
SELECT nro_factura, fecha, cod_cliente
  FROM facturas
 WHERE cod_cliente IN (1,5,7,10);

-- Solución equivalente sin usar el IN:
-- Se usan OR (Ver lógica Boole)
SELECT nro_factura,fecha,cod_cliente
  FROM facturas
 WHERE cod_cliente = 1
    OR cod_cliente = 5
    OR cod_cliente = 7
    OR cod_cliente = 10;
```

16.
```
SELECT *
  FROM plantas
 WHERE DESCRIPCION IN ('ALEGRÍA', 'ROSA', 'PINO');
```

17.
```
SELECT apellido, nombre, telefono
  FROM clientes
 WHERE apellido NOT IN ('ALVAREZ', 'PEREZ');

-- Solución equivalente sin usar el NOT IN:
-- Se usan AND (Ver lógica Boole)
SELECT apellido, nombre, telefono
  FROM clientes
 WHERE apellido <> 'ALVAREZ' AND apellido <> 'PEREZ';
```

```
-- Solución equivalente sin usar el NOT IN:
-- Se usan OR pero negado antes (Ver lógica Boole)
SELECT apellido, nombre, telefono
  FROM clientes
 WHERE NOT (apellido = 'ALVAREZ' OR apellido = 'PEREZ');
```

e) Varias condiciones de búsqueda

18.
```
SELECT *
  FROM plantas
 WHERE descripcion LIKE 'R%'
   AND precio > 7;
```

19.
```
SELECT descripcion, precio, (precio * 1.1) 'precio_aumentado'
  FROM plantas
 WHERE precio < 150
    OR precio > 800
 ORDER BY precio ASC;
```

20.
```
SELECT apellido, nombre, telefono, email
  FROM clientes
 WHERE email IS NOT NULL
    OR telefono IS NOT NULL;
```

21.
```
SELECT *, precio * stock 'Valoracion'
  FROM plantas
 WHERE ( descripcion NOT LIKE '[m,n,p]%' )
    OR ( precio > 40 AND precio * stock < 10000);

-- Solución equivalente usando ^
SELECT *, precio * stock 'Valoracion'
  FROM plantas
 WHERE ( descripcion LIKE '[^m,n,p]%' )
    OR ( precio > 40 AND precio * stock < 10000);

-- Solución equivalente no usando el rango
SELECT *, precio * stock 'Valoracion'
  FROM plantas
 WHERE (     descripcion NOT LIKE 'm%'
         AND descripcion NOT LIKE 'n%'
         AND descripcion NOT LIKE 'p%')
    OR ( precio > 40 AND precio * stock < 10000);
```

22.
```
SELECT descripcion, stock, precio,(precio * 1.15) 'precio aument'
  FROM plantas
 WHERE (stock BETWEEN 15 AND 30)
    OR ((precio * 1.15) BETWEEN 150 AND 500)
 ORDER BY descripcion;

-- Solución equivalente SIN el between
SELECT descripcion, stock, precio, (precio * 1.15) 'precio_aument'
  FROM plantas
 WHERE (stock > 15 AND stock < 30)
    OR ((precio * 1.15) > 150 AND (precio * 1.15) < 500)
 ORDER BY descripcion;
```

f) Consultas con filtros combinados

23.
```
SELECT C.APELLIDO, C.NOMBRE, CA.NOMBRE 'CALLE', C.ALTURA
  FROM CLIENTES C, CALLES CA
 WHERE C.COD_CALLE = CA.COD_CALLE;
```

24.
```
SELECT C.APELLIDO, C.NOMBRE, L.NOMBRE 'LOCALIDAD'
  FROM CLIENTES C, LOCALIDADES L
 WHERE C.COD_LOCALIDAD = L.COD_LOCALIDAD;
```

25.
```
SELECT C.APELLIDO, C.NOMBRE, B.NOMBRE 'BARRIO'
  FROM CLIENTES C, BARRIOS B
 WHERE C.COD_BARRIO = B.COD_BARRIO;
```

26.
```
SELECT F.COD_CLIENTE, F.FECHA, F.NRO_FACTURA,
       FP.DESCRIPCION 'FORMA DE PAGO'
  FROM FACTURAS F, FORMAS_PAGO FP
 WHERE F.COD_FORMA_PAGO =  FP.COD_FORMA_PAGO
   AND FP.DESCRIPCION = 'EFECTIVO'
 ORDER BY COD_CLIENTE
```

27.
```
SELECT P.COD_PLANTA, P.DESCRIPCION, P.PRECIO, P.STOCK
  FROM PLANTAS P, TIPOS_PLANTAS TP
 WHERE P.COD_TIPO_PLANTA = TP.COD_TIPO_PLANTA
   AND TP.NOMBRE = 'Flores'
```

28.
```
SELECT C.NOMBRE, C.APELLIDO, C.TELEFONO, L.NOMBRE 'LOCALIDAD'
  FROM CLIENTES C, LOCALIDADES L
 WHERE C.COD_LOCALIDAD = L.COD_LOCALIDAD
   AND L.NOMBRE = 'JESÚS MARÍA';
```

29.
```
SELECT C.NOMBRE, C.APELLIDO, CI.DESCRIPCION, L.NOMBRE 'LOCALIDAD'
  FROM CLIENTES C, CONDICIONES_IVA CI, LOCALIDADES L
 WHERE C.COD_CONDICION_IVA = CI.COD_CONDICION_IVA
   AND C.COD_LOCALIDAD = L.COD_LOCALIDAD
   AND CI.DESCRIPCION = 'MONOTRIBUTISTA'
 ORDER BY L.NOMBRE;
```

30.
```
SELECT C.NOMBRE, C.APELLIDO, C.TELEFONO, L.NOMBRE 'LOCALIDAD'
  FROM CLIENTES C, LOCALIDADES L
 WHERE C.COD_LOCALIDAD = L.COD_LOCALIDAD
   AND (L.NOMBRE = 'Córdoba' OR C.APELLIDO LIKE 'A%');
```

31.
```
SELECT C.NOMBRE, C.APELLIDO, L.NOMBRE 'LOCALIDAD'
  FROM CLIENTES C, LOCALIDADES L
 WHERE C.COD_LOCALIDAD = L.COD_LOCALIDAD
   AND L.NOMBRE IN ('CARLOS PAZ','ALTA GRACIA','ADELIA MARIA');
```

```
-- Solución equivalente sin usar el IN: Se usan OR (Ver lógica Boole)
SELECT C.NOMBRE, C.APELLIDO, L.NOMBRE 'LOCALIDAD'
  FROM CLIENTES C, LOCALIDADES L
 WHERE C.COD_LOCALIDAD = L.COD_LOCALIDAD
   AND (L.NOMBRE = 'CARLOS PAZ'
    OR L.NOMBRE = 'ALTA GRACIA'
    OR L.NOMBRE = 'ADELIA MARIA');
```

32.

```
SELECT C.NOMBRE, C.APELLIDO, C.TELEFONO, L.NOMBRE 'LOCALIDAD'
  FROM CLIENTES C, BARRIOS B, LOCALIDADES L
 WHERE C.COD_BARRIO = B.COD_BARRIO
   AND C.COD_LOCALIDAD = L.COD_LOCALIDAD
   AND L.NOMBRE NOT IN ('AREQUITO', 'CORDOBA', 'MERLO')
```

33.

```
SELECT C.NOMBRE, C.APELLIDO, B.NOMBRE 'BARRIO'
  FROM CLIENTES C, BARRIOS B, LOCALIDADES L
 WHERE C.COD_BARRIO = B.COD_BARRIO
   AND C.COD_LOCALIDAD = L.COD_LOCALIDAD
   AND (L.NOMBRE = 'CBA CAPITAL' OR C.APELLIDO NOT LIKE '[D-P]%')
 ORDER BY B.NOMBRE
```

34.

```
SELECT f.NRO_FACTURA, p.DESCRIPCION, df.CANTIDAD, df.PRECIO,
       df.CANTIDAD * df.PRECIO 'TOTAL'
  FROM FACTURAS f, DETALLES_FACTURAS df, PLANTAS p
 WHERE f.NRO_FACTURA = df.NRO_FACTURA
   AND df.COD_PLANTA = p.COD_PLANTA;
```

35.

```
SELECT C.NOMBRE, C.APELLIDO, P.NOMBRE 'PROVINCIA'
  FROM CLIENTES C, CONDICIONES_IVA CI, PROVINCIAS P
 WHERE C.COD_CONDICION_IVA = CI.COD_CONDICION_IVA
   AND C.COD_PROVINCIA = P.COD_PROVINCIA
   AND P.NOMBRE LIKE 'Co%'
   AND CI.DESCRIPCION = 'CONSUMIDOR FINAL';
```

36.

```
SELECT C.NOMBRE, C.APELLIDO, B.NOMBRE 'BARRIO', C.TELEFONO
  FROM CLIENTES C, BARRIOS b
 WHERE C.COD_BARRIO = B.COD_BARRIO
   AND B.NOMBRE IN ('General Paz', 'Centro')
   AND C.TELEFONO IS NOT NULL;
```

Solución del Problema 5: Resuelva utilizando JOIN EXPLÍCITO

1.
```
-- Solución usando LEFT JOIN;
SELECT V.NOM_VENDEDOR + ' ' + V.APE_VENDEDOR 'VENDEDOR',
       B.BARRIO 'BARRIO'
  FROM VENDEDORES V
  LEFT JOIN BARRIOS B ON V.COD_BARRIO = B.COD_BARRIO;

-- Solución equivalente usando RIGHT JOIN;
SELECT V.NOM_VENDEDOR + ' ' + V.APE_VENDEDOR 'VENDEDOR',
       B.BARRIO 'BARRIO'
  FROM BARRIOS B
 RIGHT JOIN VENDEDORES V ON B.COD_BARRIO = V.COD_BARRIO;
```

2.
```
SELECT F.NRO_FACTURA,
       F.FECHA,
       V.APE_VENDEDOR + ' '+ V.NOM_VENDEDOR 'VENDEDOR',
       C.APE_CLIENTE +' '+ C.NOM_CLIENTE 'CLIENTE'
  FROM FACTURAS F
  JOIN CLIENTES C ON F.COD_CLIENTE = C.COD_CLIENTE
  JOIN VENDEDORES V ON F.COD_VENDEDOR = V.COD_VENDEDOR
 WHERE YEAR(F.FECHA) IN (2006, 2007, 2009, 2012);
```

3.
```
SELECT F.NRO_FACTURA,
       F.FECHA,
       A.DESCRIPCION,
       DF.PRE_UNITARIO,
       DF.CANTIDAD,
       DF.PRE_UNITARIO * DF.CANTIDAD 'MONTO'
  FROM FACTURAS F
  JOIN DETALLE_FACTURAS DF ON F.NRO_FACTURA = DF.NRO_FACTURA
  JOIN ARTICULOS A ON DF.COD_ARTICULO = A.COD_ARTICULO
 WHERE MONTH(F.FECHA) = MONTH(DATEADD(MONTH, -1, GETDATE()));
```

```
-- Solución equivalente usando DATEDIFF
SELECT F.NRO_FACTURA,
       F.FECHA,
       A.DESCRIPCION,
       DF.PRE_UNITARIO,
       DF.CANTIDAD,
       DF.PRE_UNITARIO * DF.CANTIDAD 'MONTO'
  FROM FACTURAS F
  JOIN DETALLE_FACTURAS DF ON F.NRO_FACTURA = DF.NRO_FACTURA
  JOIN ARTICULOS A ON DF.COD_ARTICULO = A.COD_ARTICULO
 WHERE DATEDIFF(MONTH, F.FECHA, GETDATE()) = 1;
```

4.

```
SELECT V.COD_VENDEDOR,
       V.APE_VENDEDOR + ' '+ V.NOM_VENDEDOR 'VENDEDOR',
       F.NRO_FACTURA,
       F.FECHA,
       A.DESCRIPCION,
       DF.PRE_UNITARIO,
       DF.CANTIDAD,
       DF.PRE_UNITARIO * DF.CANTIDAD 'IMPORTE'
  FROM VENDEDORES V
  LEFT JOIN FACTURAS F ON V.COD_VENDEDOR = F.COD_VENDEDOR
                       AND YEAR(F.FECHA) = YEAR(GETDATE())-1
  LEFT JOIN DETALLE_FACTURAS DF ON F.NRO_FACTURA = DF.NRO_FACTURA
  LEFT JOIN ARTICULOS A ON DF.COD_ARTICULO = A.COD_ARTICULO;

-- Solución equivalente
SELECT V.COD_VENDEDOR,
       V.APE_VENDEDOR + ' '+ V.NOM_VENDEDOR 'VENDEDOR',
       F.NRO_FACTURA,
       F.FECHA,
       A.DESCRIPCION,
       DF.PRE_UNITARIO,
       DF.CANTIDAD,
       DF.PRE_UNITARIO * DF.CANTIDAD 'IMPORTE'
  FROM VENDEDORES V
  LEFT JOIN FACTURAS F ON V.COD_VENDEDOR = F.COD_VENDEDOR
  LEFT JOIN DETALLE_FACTURAS DF ON F.NRO_FACTURA = DF.NRO_FACTURA
  LEFT JOIN ARTICULOS A ON DF.COD_ARTICULO = A.COD_ARTICULO
 WHERE YEAR(f.fecha) = YEAR(GETDATE())-1
    OR F.FECHA IS NULL;
```

5.

```
SELECT A.DESCRIPCION,
       A.PRE_UNITARIO 'PRECIO',
       DF.CANTIDAD 'CANTIADA',
       DF.CANTIDAD * A.PRE_UNITARIO 'IMPORTE'
  FROM ARTICULOS A
  LEFT JOIN DETALLE_FACTURAS DF ON A.COD_ARTICULO = DF.COD_ARTICULO;
```

6.

```
SELECT F.NRO_FACTURA,
       F.FECHA,
       A.DESCRIPCION,
       V.APE_VENDEDOR + ' ' + V.NOM_VENDEDOR 'VENDEDOR',
       C.APE_CLIENTE + ' ' + C.NOM_CLIENTE 'CLIENTE',
       D.CANTIDAD,
       D.pre_unitario,
       D.CANTIDAD * D.pre_unitario 'IMPORTE'
  FROM FACTURAS F
  JOIN DETALLE_FACTURAS D ON F.NRO_FACTURA = D.NRO_FACTURA
  JOIN ARTICULOS A ON D.COD_ARTICULO = A.COD_ARTICULO
  JOIN VENDEDORES V ON F.COD_VENDEDOR = V.COD_VENDEDOR
  JOIN CLIENTES C ON F.COD_CLIENTE = C.COD_CLIENTE
 WHERE YEAR(F.FECHA) IN (2006, 2007)
   AND MONTH(F.FECHA) IN (2, 3)
   AND A.DESCRIPCION LIKE '[a-m]%'
 ORDER BY F.FECHA, CLIENTE, A.DESCRIPCION;
```

7.
```
SELECT C.COD_CLIENTE, C.NOM_CLIENTE, F.FECHA, F.NRO_FACTURA
  FROM CLIENTES C
  LEFT JOIN FACTURAS F ON C.COD_CLIENTE = F.COD_CLIENTE
 WHERE YEAR(F.FECHA) = 2007
    OR F.FECHA IS NULL;
```

8.
```
SELECT C.APE_CLIENTE +' '+ C.NOM_CLIENTE 'CLIENTE',
       A.DESCRIPCION,
       D.CANTIDAD * A.PRE_UNITARIO 'IMPORTE'
  FROM ARTICULOS A
  JOIN DETALLE_FACTURAS D ON A.COD_ARTICULO = D.COD_ARTICULO
  JOIN FACTURAS F ON D.NRO_FACTURA = F.NRO_FACTURA
  JOIN CLIENTES C ON F.COD_CLIENTE = C.COD_CLIENTE
 WHERE YEAR(F.FECHA) = YEAR(GETDATE())-1
   AND C.NOM_CLIENTE = 'PILAR'
   AND C.APE_CLIENTE = 'MORALES';
```

9.
```
SELECT C.APE_CLIENTE +' '+ C.NOM_CLIENTE 'CLIENTE',
       A.DESCRIPCION 'ARTICULO',
       DF.CANTIDAD 'CANTIDAD',
       (DF.CANTIDAD * A.PRE_UNITARIO) 'IMPORTE'
  FROM CLIENTES C
  JOIN FACTURAS F ON C.COD_CLIENTE = F.COD_CLIENTE
  JOIN DETALLE_FACTURAS DF ON F.NRO_FACTURA = DF.NRO_FACTURA
  JOIN ARTICULOS A ON DF.COD_ARTICULO = A.COD_ARTICULO
 WHERE C.APE_CLIENTE LIKE 'P%'
 ORDER BY CLIENTE DESC, ARTICULO DESC;
```

Atribución-No Comercial-Sin Derivadas

Se permite descargar esta obra y compartirla, siempre y cuando no sea modificado y/o alterado su contenido, ni se comercialice. Referenciarlo de la siguiente manera:
Universidad Tecnológica Nacional Facultad Regional Córdoba (S/D). Material para la Tecnicatura Universitaria en Programación, modalidad virtual, Córdoba, Argentina.